Classes and Objects

Programming and Data Structures — Week 1, Lecture 1

Dr. Innocent Nyalala

Welcome to Programming and Data Structures

Programming and Data Structures is a 15-credit core course, running in the Odd Semester from October 2026 to January 2027, with 4 lecture sessions and 2 lab sessions every week across 15 weeks. The formal prerequisite is Introduction to Programming in Python: this course assumes you can already write functions, use conditionals and loops, and read a Python error message, and builds directly on top of that foundation rather than re-teaching it. Every structure and algorithm in this course reappears later — in machine learning, in databases, in systems courses — which is why the institute treats it as foundational and weights it accordingly.

What you will be able to do by January

By the end of this course, you will be able to design Python classes using encapsulation, inheritance, and polymorphism, and explain why every data structure taught here is built as a class; implement stacks, queues, linked lists, binary trees, hash tables, and heaps entirely from first principles rather than only using library implementations; analyze the running time of an algorithm using big-O notation and use that analysis to justify choosing one structure over another; apply dynamic programming and greedy strategies to optimization problems; represent graphs and apply breadth-first search, depth-first search, and A* to reachability and shortest-path problems; and describe, with a working example, why parallel and randomized approaches are used in practice.

The 15-week roadmap

The course proceeds in this order, each week building on everything before it: Week 1, object-oriented programming and an introduction to algorithmic complexity (this week); Week 2, arrays and dynamic arrays; Week 3, stacks and queues; Week 4, linked lists; Week 5, simple sorting; Week 6, advanced sorting; Week 7, binary trees. Each of these weeks has 4 lectures and 2 lab sessions attached, matching the roadmap continued on the next slide.

The 15-week roadmap, continued

The second half of the course continues: Week 8, hash tables and heaps; Week 9, dynamic programming; Week 10, greedy algorithms; Week 11, introduction to graphs; Weeks 12–13 (a double-length module), graph algorithms including BFS, DFS, and A*; Week 14, parallel and randomized algorithms; and Week 15, project presentations and review, where the course project begun around the midpoint of the semester is presented and assessed.

How your grade is composed

Your final grade is composed of four parts. In-class assignments contribute 20 percent, made up of seven assignments completed in lab on a bi-weekly schedule. Quizzes contribute 15 percent, split across two written quizzes covering modules 1 through 5 and modules 6 through 9 respectively. The course project contributes 25 percent, broken down as a 3 percent proposal, a 5 percent first milestone, a 7 percent second milestone, a 7 percent final submission, and a 3 percent presentation — done individually or in pairs, choosing either a from-scratch machine learning algorithm or a recommendation system as the applied problem. The end-semester examination contributes the remaining 40 percent, reflecting its weight as a core course, and covers both practical and theoretical material, held in the lab.

Textbooks and how to use them

The required text is *Data Structures and Algorithms in Python* by Robert Lafore, Ami Broder, and Jeremy Canning (1st edition, Addison Wesley, 2022), which is the primary reference and from which weekly readings are assigned throughout the semester. Steven Skiena's *The Algorithm Design Manual* (2nd edition, Springer, 2008) is a reference text — turn to it once a structure already makes basic sense and you want a deeper, more mathematically thorough treatment, rather than as a first introduction to a new topic.

Policies you need to know from day one

A few policies matter from the very first week. Attendance is expected at all lectures and labs, and lab work is done during the lab session and submitted before it ends. Late assignments lose 10 percent per day late, up to three days, after which they are not accepted — but each student has three no-questions-asked late days for the semester to use at their own discretion. On integrity: discussing ideas and approaches with classmates is encouraged, but sharing or copying code is not, and every line you submit must be one you can personally explain; submissions are checked for similarity. On AI tools specifically: you may use AI coding assistants to explain concepts, read error messages, or review code you have already written yourself, but you may not use them to generate a solution you then submit —

if an assistant contributed materially to a submission, say so in a header comment. Students needing accommodations should contact the instructor within the first two weeks of the semester.

Objectives

By the end of this lecture, you will be able to write a Python class with a constructor, attributes, and at least one method; explain precisely what `self` refers to inside a method and why Python passes it automatically; and recognize the single most common mistake a first-time class author makes in `__init__`.

Outline

Today covers why a plain tuple is not good enough for structured data, how a class works as a blueprint using a `Point` example, the distinction between attributes and methods, the single mistake almost every student makes the first time they write `__init__`, a short in-class quiz, and a summary.

Why we start here, not with data structures

Every structure in this course — stack, queue, linked list, tree, graph — is built as a **Python class**. If the class mechanism is shaky, every week after this one is shaky too.

This lecture has one goal: by the end, you can look at any `class` block in this course's textbook and explain what happens when it runs.

The problem, before the solution

Suppose you are tracking points on a 2D plane and need the distance between two of them. Without a class, you might reach for a tuple:

```
p1 = (0, 0)
p2 = (3, 4)
distance = ((p1[0] - p2[0]) ** 2 + (p1[1] - p2[1]) ** 2) ** 0.5
```

This works, but `p1[0]` and `p1[1]` mean nothing without a comment, and nothing stops you from writing `p1 = (0, 0, 0)` by mistake — a 3-tuple that silently breaks this formula three lines later, far from where the bug actually is.

A class is a blueprint

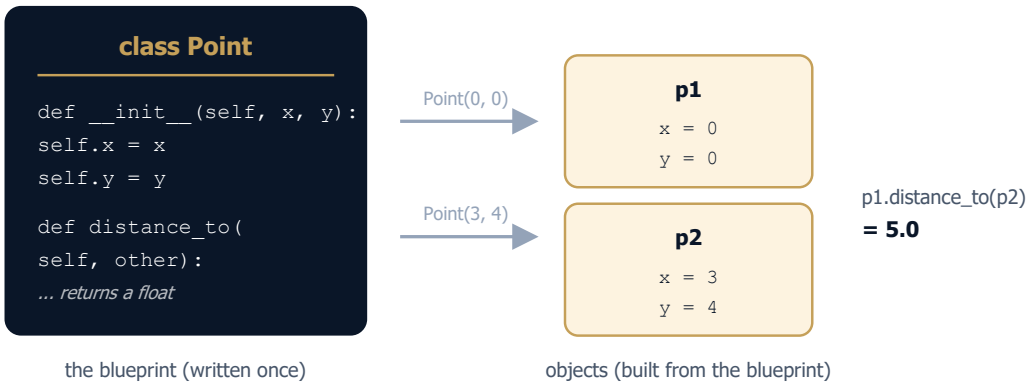
```
class Point:
    """A point in the 2D plane."""

    def __init__(self, x, y):      # runs once, when a Point is created
        self.x = x                  # self.x is THIS point's x, not some
        self.y = y                  # global variable named x

    def distance_to(self, other):
        dx = self.x - other.x
        dy = self.y - other.y
        return (dx ** 2 + dy ** 2) ** 0.5
```

`class Point:` declares the blueprint. `__init__` is the constructor — Python calls it automatically the moment you write `Point(0, 0)`. `self` is how the method reaches back into the specific object it was called on.

Class vs. object, drawn out



The class is written once. `Point(0, 0)` and `Point(3, 4)` are two different **objects**, each with its own `x` and `y`, both built from the same blueprint.

Running it

```
p1 = Point(0, 0)
p2 = Point(3, 4)
print(p1.distance_to(p2))
```

```
5.0
```

Read `p1.distance_to(p2)` as "ask `p1` to compute its distance to `p2`." `self` inside the method is `p1`; `other` is `p2`. Nothing here is special syntax — it is the same function-call mechanism as always, with the object silently passed as the first argument.

Attributes vs. methods

	Holds data	Holds behavior
Attribute	`self.x`, `self.y`	—
Method	—	`distance_to(self, other)`

An attribute is a value stored on the object. A method is a function stored on the class, called through an object. Every method's first parameter is `self` by convention — Python passes the object there automatically; you never write it yourself when calling.

Common first mistake

```
class Point:
    def __init__(self, x, y):
        x = x        # BUG: assigns to the local parameter, not self.x
        y = y

p = Point(3, 4)
print(p.x)   # AttributeError: 'Point' object has no attribute 'x'
```

Forgetting `self.` on the left-hand side is the single most common bug in a first `__init__`. `x = x` does nothing useful — it reassigns the local parameter `x` to itself and throws the value away. The object never gets an `x` attribute at all.

Live coding exercise (5 minutes)

Extend `Point` with a `midpoint(self, other)` method that returns a new `Point` sitting exactly between `self` and `other`.

```
def midpoint(self, other):
    return Point((self.x + other.x) / 2,
                 (self.y + other.y) / 2)
```

Notice the method **returns a new `Point`**, built with the same `Point(...)` constructor you already have. Methods can and often do construct more objects of their own class.

Quiz — spot the bug

```
class Rectangle:
    def __init__(self, w, h):
        w = w
        h = h
    def area(self):
        return self.w * self.h

r = Rectangle(4, 5)
print(r.area())
```

Before reading on: what happens when this code runs, and why? Work it out from what you learned this lecture, not by guessing.

Quiz — answer

This raises `AttributeError: 'Rectangle' object has no attribute 'w'`. `w = w` and `h = h` reassign the local parameters to themselves and throw the values away — exactly the bug from the `Point` example earlier, just in a new class. `area()` then reaches for `self.w`, which was never set, because the constructor never wrote to `self` in the first place.

Summary

A class is a blueprint, written once; an object is one instance built from it, with its own copy of the attributes. `__init__` runs automatically the moment an object is created, and `self` inside any method refers to that specific object. Attributes hold data (`self.x`), methods hold behavior (`def area(self)`), and the most common first-time mistake is forgetting `self.` on the left-hand side of an assignment inside `__init__`. Next lecture: what happens when we let data be changed carelessly, and how a class can refuse to allow it — encapsulation.